

User:

- 2505.12716v3.pdf

<https://github.com/wutaiqiang/Shadow-FT> maksud paper ini gimana

Kimi:

Paper **Shadow-FT** ini punya ide yang simpel tapi cerdas buat *fine-tuning* model bahasa besar (LLM), terutama model yang sudah di-*instruction-tune* (seri **Instruct**).

Intinya begini:

1. Masalah yang Ditemukan

Peneliti ini menyadari bahwa kalau kamu langsung *fine-tune* model **Instruct** (misalnya Llama-3.1-8B-Instruct atau Qwen-3-Instruct) pakai data baru, hasilnya seringkali **cuma naik dikit atau malah turun** (degradasi). Padahal model *base*-nya (yang belum di-*instruction-tune*) kalau dilatih malah bisa belajar dengan baik.

2. Observasi Penting

Mereka membandingkan bobot (weights) antara model **BASE** dan **INSTRUCT** yang berpasangan. Ternyata bedanya sangat kecil — rata-rata kurang dari 2% (bahkan untuk Llama-3.1-8B cuma ~1.8%). Artinya kedua model itu secara struktur dan bobot hampir identik.

3. Insight Utama

- **BASE model** = *good learner but weak backbone*. Dia bisa belajar pengetahuan baru dengan baik, tapi nggak pandai ngikutin instruksi.
- **INSTRUCT model** = sudah pandai ngikutin instruksi, tapi “kemampuan instruksi”-nya justru **mengganggu** proses belajar hal baru. Model ini jadi kaku (*rigid optimization*) dan resisten terhadap update bobot.

4. Solusi: Shadow-FT

Daripada *fine-tune* langsung di model Instruct, mereka mengusulkan:

1. **Fine-tune model BASE** pakai data target (bisa full-parameter atau LoRA).

2. **Hitung delta/update bobot** yang didapat BASE: $\Delta = W_BASE_baru - W_BASE_lama$

3. **Graft (tempelkan)** delta tersebut ke model INSTRUCT:

$$W_INSTRUCT_baru = W_INSTRUCT + \Delta$$

Jadi model Instruct tetap jadi “tulang punggung” yang kuat, tapi pengetahuan barunya dipelajari lewat “bayangan” (shadow)-nya yaitu model Base.

5. Keunggulan

- **Tanpa biaya tambahan:** Jumlah parameter dan biaya latih sama kayak fine-tuning biasa.
- **Mudah diimplementasikan:** Cuma beda model awal yang dilatih (Base vs Instruct), sisanya sama.
- **Konsisten lebih baik:** Di 19 benchmark (matematika, coding, reasoning), Shadow-FT selalu mengungguli fine-tuning konvensional baik pada setting full-parameter maupun LoRA.
- **Bisa dikombinasikan:** Juga berhasil dipakai bersama DPO (Shadow-DPO) dan bahkan pada model multimodal (MLLM).

Singkatnya

“Belajar di Base, eksekusi di Instruct.”

Shadow-FT memanfaatkan kenyataan bahwa Base dan Instruct bobotnya hampir sama, jadi lebih baik latih Base yang lebih “fleksibel belajar”, lalu pindahkan ilmunya ke Instruct supaya tetap jago ngikutin instruksi + punya pengetahuan baru.

Kalau mau lihat implementasinya, repo GitHub-nya ada di link yang kamu kirim:

<https://github.com/wutaiqiang/Shadow-FT>.

User:

coba cari informasi terkait metode training encoder-decoder model yang base ke instruct tuning, boleh dari arxiv atau apapun lainnya

Kimi:

Berikut rangkuman paper dan temuan terkini tentang **training encoder-decoder model dari base ke instruct tuning**, mulai dari klasik (Flan-T5) hingga riset terbaru 2025.

1. Flan-T5: Dasar Instruction Tuning pada Encoder-Decoder

Paper *Scaling Instruction-Finetuned Language Models* (Google, 2022/2023) adalah karya paling berpengaruh di sini. Intinya:

- **Base model:** T5 (encoder-decoder) yang dipre-train dengan *span corruption* pada C4.
- **Instruction tuning:** Model T5 dilatih ulang (fine-tune) pada koleksi **FLAN** — yaitu campuran 1.800+ task dengan berbagai *prompt template* dan *chain-of-thought* (CoT).
- **Hasil:** Flan-T5 mampu mengikuti instruksi zero-shot dan few-shot secara signifikan lebih baik daripada T5 base. Misalnya, Flan-T5-XL (3B) bisa mengungguli GPT-3 175B pada MMLU .

Detail training recipe Flan-T5: - **Data:** FLAN collection (Muffin, T0-SF, CoT, NIV2) dengan *maximum example cap* dan balancing. - **Hyperparameter:** Learning rate 5e-4, dropout 0.05, batch size 64, tanpa packing. Jumlah step bervariasi (98k untuk Small hingga 14k untuk XXL). - **Biaya compute:** Instruction tuning hanya membutuhkan ~0.2%–1.6% dari total biaya pre-training .

2. UL2 & Flan-UL2: Mixture of Denoisers

Paper *UL2: Unifying Language Learning Paradigms* mengusulkan pre-training objective yang lebih kaya untuk encoder-decoder:

- **Mixture of Denoisers (MoD):** Menggabungkan *span corruption*, *prefix language modeling*, dan *causal language modeling* dalam satu framework.
- **Mode switching:** Model dilatih dengan token khusus (misal [R], [S], [X]) untuk menandai mode tugas, sehingga bisa beradaptasi dengan berbagai downstream task .

Flan-UL2 adalah hasil instruction tuning UL2 dengan FLAN (serupa Flan-T5). Keunggulannya: - Tidak lagi membutuhkan *mode switch tokens* setelah extra training 100k step. - Receptive field 2048 token, lebih baik untuk few-shot learning. - Outperform Flan-T5-XXL sebesar +3.2% relatif pada NLU benchmark .

3. Encoder-Decoder Modern: RedLLM (2025)

Paper terbaru *Encoder-Decoder or Decoder-Only? Revisiting Encoder-Decoder Large Language Model* meneliti apakah arsitektur encoder-decoder masih kompetitif di era LLM modern.

- **Pre-training:** Mereka pre-train model encoder-decoder (RedLLM) dari scratch pada corpus RedPajama (1.6T token), dengan konfigurasi mirip T5/mT5.
 - **Instruction tuning:** Menggunakan FLAN collection (1.800+ task) untuk SFT, dengan input/output max 2048/512 token. Semua parameter di-update (full fine-tune).
 - **Temuan:** Encoder-decoder modern yang cukup besar tetap kompetitif dan kadang lebih efisien dibanding decoder-only untuk task tertentu .
-

4. Encoder-Decoder Gemma (2025)

Google merilis *Encoder-Decoder Gemma* yang mengadaptasi decoder-only Gemma 2 (2B & 9B) menjadi arsitektur encoder-decoder mirip mT5.

- **Adaptasi:** Decoder-only Gemma 2 diubah menjadi encoder-decoder dengan menambahkan encoder stack dan cross-attention, lalu dilanjutkan pre-training dengan **PrefixLM** dan **UL2**.
 - **Instruction tuning:** Setelah adaptasi, model menjalani **SFT (Supervised Fine-Tuning)** dan **RLHF** mengikuti recipe Gemma 2.
 - **Hasil:** Mereka menunjukkan trade-off kualitas-efisiensi yang lebih baik, terutama pada task yang memerlukan pemahaman konteks penuh sebelum generasi .
-

5. Encoder-Decoder untuk Code & Domain Spesifik

Selain umum, encoder-decoder juga dipakai untuk instruction tuning di domain sempit:

- **FlanEC**: Flan-T5 di-fine-tune untuk *Post-ASR Error Correction*, menunjukkan encoder-decoder efektif untuk task *text-to-text* dengan input panjang .
- **OpenCodeInstruct**: Meskipun fokus pada decoder-only (Llama3, Qwen2.5-Coder), mereka menunjukkan bahwa fine-tuning dari base ke instruct pada code dataset besar (5 juta sampel) bisa mengungguli model instruct bawaan pabrik . Secara konseptual, ini juga berlaku untuk encoder-decoder code model seperti CodeT5.

Ringkasan Metode Training (Base → Instruct)

Tahap	Flan-T5	UL2/Flan-UL2	Encoder-Decoder Gemma
Pre-training	Span corruption (C4)	Mixture of Denoisers (C4)	PrefixLM + UL2 (Gemma corpus)
Arsitektur	T5 encoder-decoder	T5 encoder-decoder	Gemma 2 → diadaptasi ke enc-dec
Instruction Tuning	FLAN collection (1.8k tasks)	FLAN + forget mode tokens	SFT + RLHF (Gemma recipe)
Teknik Khusus	Prompt mixing, CoT, balancing	Mode switching, longer context	Cross-attention warmup, unbalanced enc-dec
Catatan	Klasik, paling banyak	Lebih konsisten across tasks	Terbaru, menunjukkan enc-dec masih relevan

Tahap	Flan-T5	UL2/Flan-UL2	Encoder-Decoder Gemma
	dipakai		

Kesimpulan

Meskipun tren utama saat ini didominasi decoder-only (Llama, Qwen, GPT), **encoder-decoder tetap hidup dan berkembang**. Dari Flan-T5 yang membuktikan instruction tuning pada T5, UL2 yang menyatukan berbagai objective pre-training, hingga *Encoder-Decoder Gemma* dan *RedLLM* (2025) yang menunjukkan arsitektur ini masih kompetitif jika dilatih dengan recipe modern.

Jika kamu ingin implementasi praktis, repo seperti **FlashT5/FAT5** juga menyediakan implementasi T5/UL2 optimized dengan Flash Attention untuk training yang lebih efisien .

User:

tujuan utamanya adalah untuk multi-turn chatbot yang didalamnya ada task-task yang diunggulkan oleh model encoder-decoder (didalam percakapannya)

Kimi:

Berdasarkan pencarian, berikut rangkuman lengkap tentang **metode training encoder-decoder model untuk multi-turn chatbot** yang di dalamnya terdapat task-task spesifik (task-oriented dialogue).

1. Arsitektur Encoder-Decoder untuk Multi-Turn Chatbot

1.1 BERT-based Encoder-Decoder untuk Dialogue State Tracking

Paper “**A Sequence-to-Sequence Approach to Dialogue State Tracking**” (**ACL 2021**) mengusulkan **Seq2Seq-DU** yang memformalkan DST sebagai masalah sequence-to-sequence menggunakan BERT dan pointer generation .

Arsitektur: - **Utterance Encoder:** BERT-based encoder untuk mengkodekan utterances dalam dialog (current + previous utterances). - **Schema Encoder:** BERT-based encoder terpisah untuk mengkodekan deskripsi schema. -

Utterance-Schema Attender: Menghitung attention antara utterance embeddings dan schema embeddings. - **State Decoder:** LSTM dengan pointer generation untuk menghasilkan state representation.

Keunggulan untuk multi-turn: - Bisa memodelkan intents, slots, dan slot-values secara bersamaan. - Menggunakan representasi kaya dari BERT untuk utterances dan schemas. - Menangani categorical dan non-categorical slots, serta unseen schemas .

1.2 BART untuk Multi-Turn Dialogue Generation

Paper “**Multiturn dialogue generation by modeling sentence-level and discourse-level contexts**” (Nature 2022) menggunakan BART sebagai backbone dengan modifikasi :

Komponen utama: - **Bidirectional Transformer Encoder:** Mengkodekan dialogue context turn ke-T. - **Unidirectional Transformer Decoder:** Menghasilkan response. - **Difference-Aware Module:** Menghitung perbedaan informasi antara turn (T+1) dan K turn sebelumnya untuk menangkap context secara dinamis. - **Role Embedding:** Membedakan identitas speaker (EA untuk speaker pertama, EB untuk speaker kedua).

Input representation: - Token embedding + Role embedding + Position embedding (jumlah ketiganya). - [CLS] di awal setiap kalimat, [SEP] sebagai pemisah antar kalimat .

2. T5/Flan-T5 untuk Task-Oriented Dialogue

2.1 Flan-T5 sebagai Backbone TOD

Flan-T5 (Fine-tuned Language Agnostic T5) banyak digunakan untuk task-oriented dialogue karena sudah di-instruction-tune pada 1.000+ task .

Contoh implementasi multi-turn: - **Winnie (AAAI 2023):** Sistem TOD end-to-end berbasis T5 dengan: - Dialog encoder untuk memproses user utterance. - Dialogue-level Transformer layer untuk memahami histori. - Belief state decoder dan system act/response decoder terpisah .

Representasi turn:

```
H1 = T5Encoder(turn0, U1)
u0, b0, d0, a0, r0, u1 = MaxPooling(H1)
```

Di mana U=user utterance, B=belief state, D=database result, A=system act, R=response .

2.2 SynTOD: Multi-Decoder Architecture

SynTOD mengusulkan arsitektur dengan: - **Satu encoder**: Mengkodekan user input dan dialogue history. - **Dua decoder**: - Belief decoder: untuk dialogue state dan database query. - Response decoder: menghasilkan dialogue actions dan system responses .

3. Training Recipe dari Base ke Instruct untuk Multi-Turn

3.1 Pre-training Objectives

Untuk encoder-decoder modern, beberapa objective pre-training yang digunakan:

Objective	Deskripsi	Model
Span Corruption	Masking span token dan prediksi	T5, Flan-T5
Mixture of Denoisers (MoD)	Gabungan prefix LM + span corruption + causal LM	UL2, Flan-UL2
Denoising Autoencoder	Text corruption + reconstruction	BART

3.2 Instruction Tuning untuk Multi-Turn

Berdasarkan survey “**Recent Advances in LLM-Based Multi-turn Dialogue Systems**” (2025) :

Tahapan training: 1. **Pre-training**: Encoder-decoder dilatih pada corpus besar dengan objective text-to-text. 2. **Instruction Tuning**: Fine-tune pada dataset multi-turn dengan format: Input: [Dialogue History] + Current User Utterance Output: [System Response] + [Dialogue State Update] 3. **RLHF/DPO** (opsional): Alignment untuk naturalness dan task completion.

3.3 Data Format untuk Multi-Turn

Contoh format data dari paper “**Data Selection for Multi-turn Dialogue Instruction Tuning**” (2026) :

```
{
  "dialogue_history": [
    {"role": "user", "content": "..."},
    {"role": "assistant", "content": "..."}
  ],
  "current_turn": "user utterance",
  "domains": ["restaurant", "hotel"],
  "slots": {"restaurant-people": "5", "day": "Friday"}
}
```

4. Encoder-Decoder vs Decoder-Only untuk Multi-Turn Chat

4.1 Keunggulan Encoder-Decoder

Berdasarkan analisis dari Hyung Won Chung (Google/DeepMind) : -

Pemahaman input lebih baik: Encoder bisa bidirectional, memahami konteks penuh sebelum generasi. - **Pemisahan tugas jelas:** Encoder fokus understanding, decoder fokus generation. - **Cocok untuk task-oriented:** Input (dialogue history) dan output (response + state) struktur berbeda.

4.2 Tantangan Encoder-Decoder untuk Multi-Turn

- **Inefisiensi caching:** Setiap turn baru harus re-encode seluruh history karena bidirectional attention .
 - **Information bottleneck:** Di model dalam, perbedaan semantic level antara encoder final layer dan decoder first layer bisa jadi bottleneck .
-

5. Solusi Modern untuk Tantangan Multi-Turn

5.1 RTTL-DG: Real-Time Turn-Level Dialogue Generation

Paper **arXiv:2501.04877 (2025)** mengusulkan arsitektur encoder-decoder untuk spoken dialogue :

Komponen: - **Encoder:** Pre-trained HuBERT untuk speech encoding. -

Decoder: Dual role sebagai Dialogue Manager + Response Generator. - **Action**

tokens: Special tokens untuk mengontrol flow (SIL=remain silent, SPK=initiate speaking, CONT=keep speaking, STP=stop speaking).

Training: - Stage 1: ASR pre-training untuk alignment encoder-LLM. - Stage 2: Joint ASR-DST fine-tuning dengan LoRA .

5.2 Noetic State Representation (NSR) Graph

Paper **Pattern Recognition 2025** menggunakan DialoGPT sebagai encoder untuk DST : - **NSR Graph:** Representasi eksplisit dialog context yang sinkron dengan multi-turn dialogue. - **Graph Attention Network:** Untuk encoding graph. - **Sequence Decoder:** Dibimbing oleh context dari NSR Graph dan utterances.

6. Rangkuman Metode Training (Base → Instruct)

Tahap	Aktivitas	Detail
1. Pre-training	Train encoder-decoder pada corpus besar	Span corruption (T5), denoising (BART), atau MoD (UL2)
2. Instruction Tuning	Fine-tune pada dataset multi-turn	Format: history + current → response + state
3. Task-Specific Tuning	Fine-tune pada domain spesifik	DST, NLU, policy learning
4. Alignment	RLHF/DPO untuk naturalness	Preference data dari human evaluasi

Kesimpulan

Untuk **multi-turn chatbot dengan task-oriented capabilities**, encoder-decoder tetap relevan karena: 1. **Pemisahan encoder-decoder** memungkinkan pemahaman konteks yang lebih dalam sebelum generasi response. 2. **Multi-decoder architecture** (belief decoder + response decoder) memungkinkan pemisahan tugas yang jelas. 3. **Flan-T5/BART** sudah terbukti efektif dan bisa di-fine-tune untuk multi-turn dengan format sequence-to-sequence.

Namun, tantangan utama adalah **efisiensi inference** (re-encoding setiap turn) yang bisa diatasi dengan teknik caching partial atau arsitektur hybrid.

User:

- T5_GEMMA2_INSTRUCT_TRAINING_PLAN.md
- [t5gemma_2]example.py

menurut mu apalagi yang bisa ditingkatkan (mungkin metode training nya, selain sft dan dpo dll) btw cari tau tentang t5-gemma-2 serta unused token yang ada di dirinya (di tokenizernya juga, agar itu tidak muncul (karena kayak nya di usecase multi-turn gak diperlukan

Kimi:

Berikut analisis lengkap untuk kedua poinmu: **(1) metode training alternatif selain SFT & DPO**, dan **(2) T5-Gemma-2 beserta cara menangani *unused tokens*-nya** agar bersih untuk multi-turn chatbot.

1. Metode Training Selain SFT & DPO (Khusus Encoder-Decoder / Seq2Seq)

Selain SFT → DPO, ada beberapa metode alignment dan fine-tuning yang bisa kamu pertimbangkan untuk T5-Gemma-2. Banyak di antaranya **reference-free** (hemat VRAM) atau bahkan **menggabungkan SFT + alignment jadi satu tahap**.

A. ORPO — Odds Ratio Preference Optimization

ORPO menggabungkan SFT dan preference alignment dalam **satu loss function**.

- **Cara kerja:** Selama SFT, ORPO menambahkan *odds-ratio penalty* yang memaksimalkan probabilitas output *chosen* dan meminimalkan *rejected*.
- **Keunggulan:** Tidak perlu reference model, tidak perlu tahap SFT terpisah. Cocok untuk compute terbatas (A100 single GPU).

- **Untuk encoder-decoder:** Bisa dipakai di Seq2SeqTrainer dengan custom loss. TRL sudah mulai support ORPO untuk causal LM; untuk seq2seq kamu bisa adaptasi manual dengan menghitung log-prob dari decoder.

B. KTO — Kahneman-Tversky Optimization

KTO hanya butuh **sinyal biner** (output ini “bagus” atau “buruk”) untuk satu prompt, tanpa memerlukan pasangan *chosen-rejected* .

- **Cocok untuk:** Skenario di mana kamu sulit membuat pasangan preference, tapi punya banyak data dengan label kualitas (misal: respons dari API yang di-rate 1–5, lalu dikonversi jadi good/bad).
- **Keunggulan:** Dataset lebih mudah dikumpulkan; tidak perlu reference model.

C. IPO — Identity Preference Optimization

IPO menambahkan regularisasi pada DPO untuk mencegah *overfitting* ke preference data yang terbatas .

- **Kapan dipakai:** Kalau DPO-mu terlalu “agresif” dan model mulai melupakan kemampuan umum (alignment tax).

D. SimPO — Simple Preference Optimization

SimPO adalah metode **reference-free** yang menggunakan *normalized reward* berdasarkan average log-probability dan panjang respons .

- **Keunggulan:** Lebih ringan dari DPO karena tidak perlu memuat model reference sama sekali. Sangat cocok untuk single-GPU training.

E. CPO — Contrastive Preference Optimization

CPO mengganti reference model di DPO dengan *uniform prior*, sehingga lebih efisien memori .

F. SPIN — Self-Play Fine-Tuning

SPIN menggunakan *self-play*: model menghasilkan respons sendiri, lalu dilatih untuk membedakan respons buatan manusia vs buatan dirinya .

- **Cocok untuk:** Meningkatkan kualitas SFT data tanpa perlu model guru eksternal. Bisa diiterasi berkali-kali.

G. Chain of Hindsight

Selama SFT, model diberi contoh baik dan buruk secara eksplisit dalam satu sequence, lalu diajarkan untuk memilih yang baik .

- **Keunggulan:** Preference signal sudah tertanam di SFT stage; bisa mengurangi kebutuhan DPO selanjutnya.

H. RLVR — Reinforcement Learning with Verifiable Rewards

Untuk task yang punya *verifiable answer* (coding, math, structured extraction), RLVR menggunakan reward yang bisa diverifikasi secara otomatis (bukan model reward) .

- **Cocok untuk:** Chatbot yang sering menangani task terstruktur (ekstraksi entitas, checklist, kalkulasi).

I. Reflection-Tuning & FLAME

- **Reflection-Tuning:** Model merevisi dataset SFT-nya sendiri melalui introspeksi .
- **FLAME:** Alignment yang *factuality-aware*, mengurangi halusinasi .

Ringkasan Pilihan untuk T5-Gemma-2 (A100 Single GPU)

Metode	Butuh Reference Model	Pasangan Chosen/Rejected	Cocok untuk	Keterangan
ORPO	✗ Tidak	✓ Ya	Menggabung SFT+Alignment	Paling efisien waktu
KTO	✗ Tidak	✗ Tidak (binary)	Data rating/label kualitas	Mudah kumpulkan data
SimPO	✗ Tidak	✓ Ya	Hemat VRAM	Alternatif DPO ringan
IPO	✓ Ya	✓ Ya	Overfitting	Regularis

Meto de	Butuh Reference Model	Pasangan Chosen/Rejected	Cocok untuk	Keterang an
			DPO	asi tambahan
SPIN	✗ Tidak	✗ Tidak	Iterasi kualitas SFT	Self-play, tanpa guru
RLVR	✗ Tidak	✗ Tidak	Task verifiable (coding/ma th)	Reward = unit test / parser

2. T5-Gemma-2 & Unused Tokens

Apa itu T5-Gemma-2?

T5-Gemma-2 adalah keluarga model **encoder-decoder** dari Google yang diadaptasi dari Gemma 3 . Pre-trained dengan objective **UL2** (Mixture of Denoisers). Tersedia dalam ukuran: - 270m-270m - 1b-1b
- 4b-4b (yang kamu pakai)

Model ini menggunakan arsitektur T5-style (bidirectional encoder + autoregressive decoder) dengan tied embeddings, RoPE, dan context window hingga **131K token** .

Unused Tokens di Tokenizer

T5-Gemma-2 (dan Gemma pada umumnya) menyediakan **99 unused tokens** (<unused0> sampai <unused98>) di awal vocabulary .

Fungsi asli: Di model UL2, token-token ini dipakai sebagai **sentinel tokens** untuk masking span dalam objective denoising (mirip <extra_id_0> di T5 asli) . Mereka sudah terintegrasi di added_tokens_decoder dan vocabulary utama.

Masalah untuk chatbot multi-turn: Jika model belum sepenuhnya “paham” bahwa unused tokens ini bukan bagian dari output chat, ada risiko model menghasilkan token seperti <unused23> atau <unused45> di tengah percakapan.

3. Cara Menghilangkan / Menekan Unused Tokens di Output

Ada beberapa strategi, dari yang paling sederhana hingga yang permanen:

Opsi 1: `bad_words_ids` saat Generate (Cepat, Non-destruktif)

Paling mudah dan **tidak mengubah model/tokenizer**. Cukup beri tahu

`model.generate()` untuk tidak pernah menghasilkan token-token tersebut.

```
# Dapatkan semua ID unused token
unused_ids = []
for i in range(99):
    token_str = f"<unused{i}>"
    token_id = tokenizer.convert_tokens_to_ids(token_str)
    if token_id != tokenizer.unk_token_id:
        unused_ids.append([token_id]) # bad_words_ids butuh list of list

# Saat generate
outputs = model.generate(
    **inputs,
    max_new_tokens=512,
    bad_words_ids=unused_ids, # tekan semua unused tokens
    eos_token_id=tokenizer.convert_tokens_to_ids("<end_of_turn>"),
)
```

Catatan: `bad_words_ids` di HuggingFace menerima *list of list* karena bisa mencekal multi-token phrases .

Opsi 2: `suppress_tokens` di `GenerationConfig`

Kamu bisa membuat `GenerationConfig` khusus dan set `suppress_tokens`:

```
from transformers import GenerationConfig

gen_config = GenerationConfig.from_pretrained(model_id)
gen_config.suppress_tokens = unused_ids # list of int, bukan list of list
gen_config.eos_token_id = tokenizer.convert_tokens_to_ids("<end_of_turn>")

# Simpan ke model
model.generation_config = gen_config
```

Ini lebih bersifat “global” untuk model tersebut.

Opsi 3: Ganti Unused Tokens dengan Special Chat Tokens (Permanen)

Kalau kamu ingin memanfaatkan slot vocabulary yang sudah ada (tanpa menambah vocab size), kamu bisa **“rebrand”** unused tokens jadi token khusus untuk chat formatmu .

Contoh: Ganti <unused0> jadi <|turn_start|>, <unused1> jadi <|turn_end|>, dsb.

```
# Mapping: old -> new
replacements = {
    "<unused0>": "<|turn_start|>",
    "<unused1>": "<|turn_end|>",
    # ... dst
}

# Update tokenizer vocabulary mapping
for old, new in replacements.items():
    old_id = tokenizer.convert_tokens_to_ids(old)
    # Tambahkan sebagai token baru (tapi sebenarnya kita reuse ID)
    tokenizer.added_tokens_decoder[old_id] = new
    tokenizer.vocab[new] = old_id
    # Hapus old dari vocab dict jika perlu
    if old in tokenizer.vocab:
        del tokenizer.vocab[old]

# Simpan tokenizer yang sudah dimodifikasi
tokenizer.save_pretrained("./t5gemma-chat-tokenizer")
```

Keuntungan: Tidak perlu resize embedding model (karena vocab size tetap).

Token baru langsung punya embedding yang sudah terlatih (meski semula untuk UL2, jadi tetap perlu fine-tuning agar maknanya sesuai).

Opsi 4: Masking Logits (Paling “Kuat”)

Jika kamu ingin benar-benar memastikan probabilitas unused tokens = 0, kamu bisa override logits processor:

```
from transformers import LogitsProcessor

class SuppressTokensLogitsProcessor(LogitsProcessor):
    def __init__(self, suppress_tokens):
        self.suppress_tokens = suppress_tokens

    def __call__(self, input_ids, scores):
        scores[:, self.suppress_tokens] = -float("inf")
        return scores

# Pakai saat generate
```



```
processor = SuppressTokensLogitsProcessor(unused_ids)
outputs = model.generate(**inputs, logits_processor=[processor])
```

4. Rekomendasi Praktis untuk Setup-mu

Berdasarkan kendala compute (A100 single GPU, ~16GB VRAM untuk 4B) dan tujuan multi-turn chatbot Indonesia:

Aspek	Rekomendasi
Unused tokens	Pakai Opsi 1 (bad_words_ids) dulu untuk smoke test. Kalau sudah yakin, beralih ke Opsi 3 (rebrand unused tokens) untuk format chat yang lebih bersih.
Training pipeline	SFT (1 epoch) → ORPO atau KTO (1 epoch). ORPO paling efisien karena gabung SFT+alignment. KTO kalau data preference-mu berbentuk rating, bukan pasangan.
Data format	Flatten multi-turn dengan <start_of_turn> / <end_of_turn> (format Gemma). Pastikan system prompt konsisten.
LoRA	Target modules: q/k/v/o, gate/up/down, r=32, alpha=64 (sesuai rencanamu). Untuk ORPO/KTO, loss function perlu custom atau cari implementasi di TRL yang support seq2seq.
Evaluasi	Simpan checkpoint tiap fraction epoch; pilih yang terbaik berdasarkan fluency + task accuracy, bukan epoch terakhir.

Apakah kamu ingin saya buat contoh kode custom trainer untuk ORPO pada T5Gemma2ForConditionalGeneration dengan LoRA?

User:

btw masalah dataset untuk rekomendasi-rekomendasi yang kamu bilang ini format datasetnya gimana?

Kimi:

Berikut format dataset untuk masing-masing metode training yang saya sebutkan, disesuaikan untuk **encoder-decoder (seq2seq)** dan **multi-turn chatbot** dengan format Gemma (<start_of_turn>, <end_of_turn>).

1. SFT (Supervised Fine-Tuning) — Dasar

Format Dataset (Nested → Flatten)

Raw (nested, sebelum unroll):

```
{
  "conversations": [
    {"role": "system", "content": "Kamu adalah asisten AI yang membantu ..."},
    {"role": "user", "content": "Halo, apa kabar?"},
    {"role": "assistant", "content": "Halo! Saya baik, terima kasih. Ada yang bisa saya bantu?"},
    {"role": "user", "content": "Bisa ringkas teks ini? [teks panjang.. .]"},
    {"role": "assistant", "content": "Tentu! Berikut ringkasan: ..."}
  ]
}
```

Flattened (masuk ke Seq2SeqTrainer):

Baris 1:

```
{
  "input": "<start_of_turn>user\nHalo, apa kabar?<end_of_turn>\n<start_of_turn>model\n",
  "target": "Halo! Saya baik, terima kasih. Ada yang bisa saya bantu?<end_of_turn>"
}
```

Baris 2:

```
{
  "input": "<start_of_turn>user\nHalo, apa kabar?<end_of_turn>\n<start_of_turn>model\nHalo! Saya baik, terima kasih. Ada yang bisa saya bantu?<end_of_turn>\n<start_of_turn>user\nBisa ringkas teks ini? [teks panjang...]<end_of_turn>\n<start_of_turn>model\n",
  "target": "Tentu! Berikut ringkasan: ...<end_of_turn>"
}
```

Catatan: System prompt bisa dimasukkan di awal setiap input, atau di-skip jika model sudah cukup mengerti dari training.

2. DPO (Direct Preference Optimization)

Format Dataset

```
{
  "prompt": "<start_of_turn>user\\nBisa terjemahkan ini ke Indonesia? 'The quick brown fox jumps over the lazy dog.'<end_of_turn>\\n<start_of_turn>model\\n",
  "chosen": "Rubah coklat cepat melompat di atas anjing yang malas.<end_of_turn>",
  "rejected": "Rubah coklat cepat melompat di atas anjing malas.<end_of_turn>"
}
```

Aturan: - prompt = semua turn sebelum respons asisten (termasuk history multi-turn). - chosen = respons yang lebih baik (manusia pilih / API kualitas tinggi). - rejected = respons yang lebih buruk (model lama / API kualitas rendah / respons singkat tidak lengkap).

Multi-turn dengan history:

```
{
  "prompt": "<start_of_turn>user\\nHalo<end_of_turn>\\n<start_of_turn>model\\nHalo! Ada yang bisa dibantu?<end_of_turn>\\n<start_of_turn>user\\nTerjemahkan 'Hello world' ke Indonesia<end_of_turn>\\n<start_of_turn>model\\n",
  "chosen": "Halo dunia<end_of_turn>",
  "rejected": "Halo<end_of_turn>"
}
```

3. ORPO (Odds Ratio Preference Optimization)

Format Dataset — Gabungan SFT + Preference dalam Satu Baris

```
{
  "prompt": "<start_of_turn>user\\nJelaskan photosynthesis dengan sederhana<end_of_turn>\\n<start_of_turn>model\\n",
  "chosen": "Fotosintesis adalah proses tanaman membuat makanan sendiri menggunakan sinar matahari, air, dan karbon dioksida. Hasilnya adalah glukosa dan oksigen.<end_of_turn>",
  "rejected": "Fotosintesis adalah proses tanaman.<end_of_turn>"
}
```

Bedanya dengan DPO: - ORPO tidak butuh reference model. - Loss ORPO = SFT loss + odds ratio penalty. - Dataset sama persis dengan DPO, tapi training jadi **satu tahap** (bisa mulai dari base model langsung, tanpa SFT terpisah).

Untuk T5-Gemma-2 seq2seq, ORPO menghitung: - `log_prob_chosen` dari decoder pada chosen sequence. - `log_prob_rejected` dari decoder pada rejected sequence. - Odds ratio = $\log(\text{chosen} / \text{rejected})$.

4. KTO (Kahneman-Tversky Optimization)

Format Dataset — **Hanya Butuh Binary Label**

```
{
  "prompt": "<start_of_turn>user\nApa ibu kota Indonesia?<end_of_turn>\n<start_of_turn>model\n",
  "completion": "Ibu kota Indonesia adalah Jakarta.<end_of_turn>",
  "label": true
}

{
  "prompt": "<start_of_turn>user\nApa ibu kota Indonesia?<end_of_turn>\n<start_of_turn>model\n",
  "completion": "Ibu kota Indonesia adalah Surabaya.<end_of_turn>",
  "label": false
}
```

Aturan: - `label: true` = respons ini “bagus” / desirable. - `label: false` = respons ini “buruk” / undesirable. - **Tidak perlu pasangan** untuk prompt yang sama! Kamu bisa punya 1000 baris `true` dan 500 baris `false` dari prompt berbeda-beda.

Konversi dari data rating:

```
# Rating 4-5 → true, Rating 1-2 → false, Rating 3 → buang
if rating >= 4:
    label = True
elif rating <= 2:
    label = False
# rating 3 di-skip
```

5. SimPO (Simple Preference Optimization)

Format Dataset — Sama dengan DPO

```
{
  "prompt": "<start_of_turn>user\nTulis puisi pendek tentang hujan<end_of_turn>\n<start_of_turn>model\n",
  "chosen": "Hujan turun perlahan\nMembasahi bumi yang haus\nDaun-daun menari gembira\nAroma tanah menyejukkan jiwa<end_of_turn>",
  "rejected": "Hujan turun. Basah. Selesai.<end_of_turn>"
}
```

Bedanya dengan DPO: - SimPO menggunakan **normalized reward**

berdasarkan average log-probability per token dan panjang sequence. - Formula:
$$\text{reward} = (\text{sum_log_prob} / \text{length}) - \text{gamma}$$
, di mana gamma adalah konstanta.

6. SPIN (Self-Play Fine-Tuning)

Format Dataset — Butuh Respons dari Model Sendiri

Tahap 1: Generate respons dari model saat ini

```
# Model saat ini (checkpoint terakhir) generate respons untuk prompt
outputs = model.generate(prompts)
```

Tahap 2: Label dengan bantuan manusia / API / heuristik

```
{
  "prompt": "<start_of_turn>user\nCeritakan tentang wayang kulit<end_of_turn>\n<start_of_turn>model\n",
  "human_response": "Wayang kulit adalah seni pertunjukan tradisional Indonesia yang menggunakan boneka datar dari kulit kerbau...<end_of_turn>",
  "model_response": "Wayang kulit adalah boneka dari kulit. Dari Jawa.<end_of_turn>",
  "label": "human" # human lebih baik
}
```

Training: Model dilatih untuk membedakan human_response (chosen) vs model_response (rejected) — mirip DPO tapi rejected-nya dari dirinya sendiri.

7. RLVR (Reinforcement Learning with Verifiable Rewards)

Format Dataset — Task dengan Jawaban yang Bisa Diverifikasi

```
{
  "prompt": "<start_of_turn>user\nHitung 15 * 24 + 8<end_of_turn>\n<start_of_turn>model\n",
  "chosen": "360 + 8 = 368",
  "rejected": "360 + 8 = 368"
}
```

```

    "completion": "15 * 24 = 360, 360 + 8 = 368. Jadi jawabannya adalah 368.<end_of_turn>",
    "verifier_result": true,
    "extracted_answer": "368"
}

```

Verifikasi otomatis:

```

def verify_math(answer, ground_truth):
    return eval(answer.strip()) == ground_truth

def verify_code(code, test_cases):
    # Jalankan code, cek output vs expected
    pass

def verify_extraction(text, expected_entities):
    # Cek entity extraction dengan parser
    pass

```

Reward function: - reward = 1.0 jika verifikasi lolos. - reward = 0.0 jika gagal. - Bisa juga partial reward untuk jawaban hampir benar.

8. Ringkasan Format per Metode

Metode	Kolom Wajib	Bentuk Label	Butuh Reference Model
SFT	input, target	-	✗
DPO	prompt, chosen, rejected	Pasangan	✓
ORPO	prompt, chosen, rejected	Pasangan	✗
KTO	prompt, completion, label	Binary (true/false)	✗
SimPO	prompt, chosen, rejected	Pasangan	✗
SPIN	prompt, human_response, model_response	Pasangan (self-play)	✗

Metode	Kolom Wajib	Bentuk Label	Butuh Reference Model
RLVR	prompt, completion, verifier_result	Boolean / float	✗

9. Contoh Konversi dari Dataset Nested-mu

Asumsi kamu punya file `t5-gemma-2-chat-instruct-merged.jsonl` dengan format nested:

```
import json

def flatten_for_sft(conv_thread):
    """Ubah nested conversation jadi baris-baris SFT."""
    turns = conv_thread["conversations"]
    system = ""
    # Ambil system prompt jika ada
    if turns and turns[0]["role"] == "system":
        system = turns[0]["content"]
        turns = turns[1:]

    rows = []
    history = ""

    for i in range(0, len(turns), 2):
        user_turn = turns[i]
        assistant_turn = turns[i+1] if i+1 < len(turns) else None

        user_content = user_turn["content"]

        # Build input: system (optional) + history + current user + model prefix
        input_text = f"{history}<start_of_turn>user\n{user_content}<end_of_turn>\n<start_of_turn>model\n"

        if assistant_turn:
            target = assistant_turn["content"] + "<end_of_turn>"
            rows.append({"input": input_text, "target": target})
            # Update history untuk turn berikutnya
            history += f"<start_of_turn>user\n{user_content}<end_of_turn>\n<start_of_turn>model\n{target}\n"

    return rows
```

```
def convert_to_dpo_orpo(conv_thread, bad_response_generator):  
    """Ubah jadi format DPO/ORPO dengan generate rejected response."""  
    # ... implementasi serupa, tapi generate rejected dari model lama/h  
euristik  
    pass  
  
def convert_to_kto(conv_thread, rating):  
    """Ubah jadi format KTO dengan rating."""  
    # ...  
    pass
```

Apakah kamu ingin saya buat script Python lengkap untuk konversi dari format nested-mu ke masing-masing format (SFT, DPO, ORPO, KTO) sekaligus?
